

EXP 4 Hashing sha 256

Server.java

```
import java.io.*;
import java.net.*;
import java.security.MessageDigest;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;

public class Server {

    static final String SECRET_KEY = "1234567890123456"; // 16 chars for AES

    public static void main(String[] args) throws Exception {

        ServerSocket ss = new ServerSocket(5000);
        System.out.println("Server Started...");
        System.out.println("Waiting for client...");

        Socket s = ss.accept();
        System.out.println("Client Connected.");

        PrintWriter out = new PrintWriter(s.getOutputStream(), true);

        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));

        System.out.print("Enter Exam Paper Message: ");
        String message = br.readLine();

        String hash = sha256(message);
        String encryptedMessage = encryptAES(message);

        out.println(encryptedMessage);
        out.println(hash);

        System.out.println("\nOriginal Message: " + message);
        System.out.println("SHA-256 Hash: " + hash);
        System.out.println("Encrypted Message Sent: " + encryptedMessage);

        s.close();
        ss.close();
    }

    static String encryptAES(String message) throws Exception {

        SecretKeySpec key = new SecretKeySpec(SECRET_KEY.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance("AES");
        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] encryptedBytes = cipher.doFinal(message.getBytes());
```

```

        return Base64.getEncoder().encodeToString(encryptedBytes);
    }

    static String sha256(String message) throws Exception {

        MessageDigest md = MessageDigest.getInstance("SHA-256");

        byte[] hashBytes = md.digest(message.getBytes());

        StringBuilder sb = new StringBuilder();

        for (byte b : hashBytes) {
            sb.append(String.format("%02x", b));
        }

        return sb.toString();
    }
}

```

Client.java

```

import java.io.*;
import java.net.*;
import java.security.MessageDigest;
import java.util.Base64;
import javax.crypto.Cipher;
import javax.crypto.spec.SecretKeySpec;

public class Client {

    static final String SECRET_KEY = "1234567890123456"; // same 16 chars key

    public static void main(String[] args) throws Exception {

        Socket s = new Socket("localhost", 5000);

        BufferedReader in = new BufferedReader(new InputStreamReader(s.getInputStream()));

        String encryptedMessage = in.readLine();
        String receivedHash = in.readLine();

        String decryptedMessage = decryptAES(encryptedMessage);
        String newHash = sha256(decryptedMessage);

        System.out.println("Encrypted Message Received: " + encryptedMessage);
        System.out.println("Decrypted Message: " + decryptedMessage);
        System.out.println("Received Hash: " + receivedHash);
        System.out.println("New Hash: " + newHash);

        if (receivedHash.equals(newHash)) {
            System.out.println("\nIntegrity Verified: Message is NOT modified");
        } else {

```

```

        System.out.println("\nIntegrity Failed: Message is modified");
    }

    s.close();
}

static String decryptAES(String encryptedMessage) throws Exception {

    SecretKeySpec key = new SecretKeySpec(SECRET_KEY.getBytes(), "AES");

    Cipher cipher = Cipher.getInstance("AES");
    cipher.init(Cipher.DECRYPT_MODE, key);

    byte[] decodedBytes = Base64.getDecoder().decode(encryptedMessage);
    byte[] decryptedBytes = cipher.doFinal(decodedBytes);

    return new String(decryptedBytes);
}

static String sha256(String message) throws Exception {

    MessageDigest md = MessageDigest.getInstance("SHA-256");

    byte[] hashBytes = md.digest(message.getBytes());

    StringBuilder sb = new StringBuilder();

    for (byte b : hashBytes) {
        sb.append(String.format("%02x", b));
    }

    return sb.toString();
}
}

```